Новое ТЗ РГР

📄 1. Общее описание проекта

1.1 Название проекта

Онлайн-игра «Покер: Техасский Холдем».

1.2 Цель разработки

Целью разработки является создание многопользовательского программного приложения, реализующего игру в покер по правилам Техасского Холдема с использованием клиент-серверной архитектуры на языке программирования Java.

Разрабатываемая система должна обеспечивать:

- взаимодействие нескольких пользователей в режиме реального времени;
- корректную реализацию игровой логики;
- обработку действий игроков;
- синхронизацию состояния игры между клиентами.

Дополнительно проект направлен на:

- закрепление навыков объектно-ориентированного программирования;
- освоение сетевого программирования (работа с сокетами);
- реализацию многопользовательских систем.

1.3 Краткое описание игры

Разрабатываемое приложение представляет собой карточную игру «Техасский Холдем».

Игровой процесс включает:

- раздачу каждому игроку двух закрытых карт;
- поэтапное открытие пяти общих карт на столе;
- проведение раундов ставок между игроками;
- определение победителя на основе лучшей комбинации карт.

Игра проходит в несколько последовательных этапов:

- префлоп;
- флоп;
- тёрн;
- ривер;
- шоудаун.

Победителем считается игрок, собравший наиболее сильную комбинацию из пяти карт (две карты на руках и три карты со стола).

1.4 Особенности реализации

Данное программное изделие имеет следующие особенности:

- реализуется **один игровой стол** (без системы комнат);
- поддерживается от **2 до 4 игроков**;
- игра начинается автоматически при наличии минимального количества игроков;
- все вычисления и управление игровым процессом выполняются на сервере;
- клиенты выполняют функцию отображения состояния игры и передачи действий пользователя;

Дополнительно:

- отсутствует система регистрации и авторизации (используется ввод имени);

- отсутствует система сохранения прогресса;
- игровой процесс является непрерывным (после окончания раздачи начинается новая).

1.5 Используемые технологии

В процессе разработки используются следующие технологии и средства:

1.5.1 Язык программирования

- Java (версия 8 и выше)

1.5.2 Сетевое взаимодействие

- TCP-соединение
- библиотека `java.net` (Socket, ServerSocket)

1.5.3 Формат обмена данными

- JSON (для передачи сообщений между клиентом и сервером)

1.5.4 Архитектура

- клиент-серверная модель
- разделение логики и интерфейса

1.5.5 Пользовательский интерфейс

- JavaFX или Swing (в зависимости от реализации)

1.6 Ограничения проекта

В рамках данной разработки принимаются следующие ограничения:

- отсутствие поддержки более 4 игроков;
- отсутствие нескольких игровых столов;
- отсутствие сложной системы ставок (например side-pot может быть упрощен);
- отсутствие криптографической защиты соединения;
- ограничение на использование стандартных библиотек Java (без внешних фреймворков);

📄 2. Описание игрового процесса

2.1 Общие правила игры

Разрабатываемая система реализует карточную игру «Техасский Холдем», являющуюся одной из наиболее распространённых разновидностей покера.

В игре используется стандартная колода из 52 карт без джокеров. Каждый игрок получает две закрытые карты (карты «на руках»), после чего на стол поэтапно выкладываются пять общих карт.

Игроки формируют комбинацию из пяти карт, используя:

- любые две/одну карты на руках;
- любые три/четыре карты со стола.

Победителем считается игрок, собравший наиболее сильную комбинацию в соответствии с установленной иерархией покерных комбинаций.

2.2 Цель игры

Целью игры является:

- выигрыш банка (pot), сформированного из ставок всех игроков;
- достижение максимального количества фишек в процессе последовательных раздач.

Игрок может выиграть:

- за счёт наличия сильнейшей комбинации;
- за счёт того, что все остальные игроки сбросили карты (Fold).

2.3 Количество игроков

В игре участвуют:

- минимум: 2 игрока;
- максимум: 4 игрока.

Особенности:

- игра не начинается, пока не подключено менее 2 игроков;
- при подключении более 4 игроков новые подключения отклоняются;
- при выходе игрока во время игры его участие прекращается.

2.4 Структура раздачи

Каждая раздача представляет собой полный игровой цикл и состоит из нескольких последовательных этапов.

2.4.1 Префлоп

На этапе префлопа выполняются следующие действия:

1. Формируется и перемешивается колода.
2. Каждому игроку раздаются по 2 карты (в закрытую).
3. Назначаются блайнды (при необходимости упрощённой реализации — фиксированные ставки).
4. Начинается первый раунд ставок.

Особенности:

- карты игроков не видны другим участникам;
- игроки принимают решения на основе своих карт.

2.4.2 Флоп

На этапе флопа:

1. На стол выкладываются 3 общие карты (открытые).
2. Все игроки получают доступ к этим картам.
3. Начинается второй раунд ставок.

Особенности:

- комбинации начинают формироваться с учётом общих карт;
- стратегия игры меняется в зависимости от выпавших карт.

2.4.3 Тёрн

На этапе тёрна:

1. На стол выкладывается 4-я общая карта.
2. Начинается третий раунд ставок.

Особенности:

- увеличивается точность оценки комбинации;

- возрастает вероятность сильных комбинаций.

2.4.4 Ривер

На этапе ривера:

1. На стол выкладывается 5-я (последняя) карта.
2. Проводится финальный раунд ставок.

Особенности:

- все карты известны;
- игроки принимают окончательные решения.

2.4.5 Шоудаун

Шоудаун — финальный этап раздачи.

Выполняются следующие действия:

1. Все оставшиеся в игре игроки вскрывают карты.
2. Для каждого игрока определяется лучшая комбинация.
3. Выполняется сравнение комбинаций.
4. Определяется победитель (или несколько победителей).
5. Банк распределяется между победителями.

2.5 Раунды ставок

На каждом этапе (кроме раздачи) выполняется раунд ставок.

2.5.1 Общая логика ставок

- игроки ходят по очереди;
- порядок ходов фиксирован;
- каждый игрок должен:

- либо уравнять ставку;
- либо повысить ставку;
- либо выйти из игры;

2.5.2 Завершение раунда ставок

Раунд ставок заканчивается, если:

- все игроки уравняли ставки;
- все игроки, кроме одного, сбросили карты;
- все игроки сделали Check (если нет активной ставки);

2.6 Возможные исходы раздачи

Раздача может завершиться следующими способами:

2.6.1 Победа без вскрытия

Если все игроки, кроме одного, сделали Fold:

- оставшийся игрок автоматически выигрывает банк;

2.6.2 Победа по комбинации

Если после ривера в игре осталось несколько игроков:

- проводится шоудаун;
- определяется победитель по комбинации;

2.6.3 Ничья

Если у нескольких игроков одинаковые комбинации:

- банк делится между ними;

2.7 Повторение игрового цикла

После завершения раздачи:

1. Балансы игроков обновляются;
2. Проверяется наличие активных игроков;
3. При наличии 2+ игроков:
    - автоматически начинается новая раздача;

Игровой процесс является непрерывным.

📄 3. Пользовательские сценарии

3.1 Общая характеристика взаимодействия

Пользователь взаимодействует с системой через клиентское приложение, подключаясь к серверу и принимая участие в игровом процессе.

Все действия пользователя:

- выполняются через графический интерфейс;
- отправляются на сервер;
- обрабатываются сервером;
- результат возвращается клиенту в виде обновленного состояния игры.

3.2 Сценарий подключения игрока

3.2.1 Запуск клиента

Пользователь запускает клиентское приложение.

3.2.2 Ввод имени

Пользователю предлагается ввести имя (никнейм).

Требования к вводу:

- длина: от 3 до 16 символов;
- допустимые символы:
  - латинские буквы (A–Z, a–z);
  - цифры (0–9);
  - символ « _ »;

Ограничения:

- запрещены пробелы;
- запрещены специальные символы;
- запрещены пустые строки;

3.2.3 Проверка имени

Проверка осуществляется в два этапа:

1. **На клиенте**:
   - проверка длины;
   - проверка допустимых символов;
2. **На сервере**:
   - проверка уникальности имени среди подключенных игроков;

3.2.4 Установление соединения

После успешного ввода имени:

1. клиент устанавливает TCP-соединение с сервером;
2. отправляет сообщение подключения;

3. сервер добавляет игрока в список;
4. клиент получает подтверждение подключения;

3.2.5 Ошибки подключения

Возможные ошибки:

- сервер недоступен;
- превышено максимальное количество игроков;
- имя уже занято;

В этих случаях пользователю выводится сообщение об ошибке.

3.3 Сценарий ожидания игроков

После подключения:

- игрок попадает в состояние ожидания;
- отображается количество подключенных игроков;
- игра не начинается, пока не подключено минимум 2 игрока;

3.4 Сценарий начала игры

Когда количество игроков ≥ 2:

1. сервер инициирует новую раздачу;
2. всем игрокам отправляется состояние начала игры;
3. каждому игроку раздаются карты;
4. начинается первый раунд ставок;

3.5 Сценарий выполнения хода

3.5.1 Получение хода

Когда наступает очередь игрока:

- интерфейс активирует кнопки действий;
- запускается таймер хода;

3.5.2 Выбор действия

Игрок выбирает одно из доступных действий:

- Fold
- Check
- Call
- Raise
- All-in

3.5.3 Отправка действия

После выбора:

- действие отправляется на сервер;
- сервер проверяет корректность;
- при успехе действие применяется;

3.5.4 Обработка ошибки

Если действие некорректно:

- сервер отклоняет его;
- клиент получает уведомление;
- игрок должен выбрать другое действие;

3.6 Сценарий таймаута

Если игрок не сделал ход за отведённое время:

- если возможно действие Check → выполняется Check;
- иначе → выполняется Fold;

Игрок уведомляется о выполненном автоматическом действии.

3.7 Сценарий завершения раздачи

После окончания раунда:

1. отображаются комбинации игроков;
2. определяется победитель;
3. банк распределяется;
4. отображаются результаты;

3.8 Сценарий начала новой раздачи

После завершения:

- игра автоматически переходит к новой раздаче;
- игроки продолжают участие без повторного подключения;

3.9 Сценарий выхода игрока

3.9.1 Добровольный выход

Игрок может закрыть приложение:

- соединение разрывается;
- сервер удаляет игрока;

3.9.2 Выход во время игры

Если игрок выходит во время раздачи:

- выполняется автоматический Fold;
- игрок исключается из текущей игры;

3.9.3 Последствия

Если остается менее 2 игроков:

- игра останавливается;
- система возвращается в режим ожидания;

📄 4. Функциональные требования

4.1 Общие положения

Функциональные требования определяют поведение системы, включая:

- обработку действий игроков;
- управление игровым процессом;
- взаимодействие клиента и сервера;
- обработку данных и состояний игры.

Система реализуется по принципу:
👉 **сервер управляет логикой, клиент — отображает**

4.2 Работа с пользователем

4.2.1 Ввод имени пользователя

Требования к имени:

- длина: от 3 до 16 символов;
- допустимые символы:
  - латиница (A–Z, a–z);
  - цифры (0–9);
  - символ « _ »;

Ограничения:

- запрещены пробелы;
- запрещены спецсимволы;
- запрещена кириллица;

Проверка:

На клиенте:

- regex-проверка:

^ [a-zA-Z0-9_]{3,16}$

На сервере:

- проверка уникальности имени;

4.2.2 Подключение к серверу

Алгоритм:

1. Клиент создает соединение:
2. Инициализирует потоки:
3. Отправляет пакет о подключении:
4. Сервер:

- проверяет имя;
- добавляет игрока;
- возвращает подтверждение;

4.3 Управление игроками

4.3.1 Хранение игроков

Производится в списке

4.3.2 Ограничения

- минимум: 2
- максимум: 4

4.3.3 Отключение игрока

При отключении:

- соединение закрывается;
- игрок удаляется;
- выполняется авто-Fold;

4.4 Игровая логика

4.4.1 Игровой цикл

- Дождались подключения 2 игроков
- Начали раунд

4.4.2 Этапы игры

- PRE_FLOP
- FLOP
- TURN
- RIVER
- SHOWDOWN

4.5 Работа с картами

4.5.1 Структура карты

- Масть
- Значение

4.5.2 Колода

- 52 карты;
- создается при каждой раздаче;

4.5.3 Перемешивание

- Рандомизация колоды происходит путем использования класса java.util.Random

---

4.5.4 Раздача

- игрокам: по 2 карты из колоды, карты уникальные
- стол:

- 3 карты (флоп)
- 1 карта (тёрн)
- 1 карта (ривер)

4.6 Действия игрока

4.6.1 Перечень

- Fold
- Check
- Call
- Raise
- All-in

4.6.2 Проверка действий

Сервер проверяет:

- Очередность
- Наличие фишек

Логика ставок:

- Check только без ставки
- Raise ≥ минимального

4.7 Таймер хода

4.7.1 Параметры

- 90 секунд

4.7.2 Реализация

ScheduledExecutorService

4.7.3 Таймаут

Если игрок не ходит:

- если можно → Check
- иначе → Fold

4.8 Ставки и банк

4.8.1 Формирование

pot += bet;

4.8.2 All-in

- игрок ставит всё;
- участвует до конца раздачи;

4.9 Комбинации

4.9.1 Алгоритм

1. берутся 7 карт

2. генерируются 21 комбинация
3. выбирается лучшая

4.9.2 Рейтинг

ROYAL_FLUSH = 10
...
HIGH_CARD = 1

4.10 Определение победителя

4.10.1 Логика

- сравнение комбинаций
- сравнение старших карт

4.10.2 Ничья

- несколько победителей

4.11 Распределение банка

4.11.1 Деление

share = pot / winners.size();

4.11.2 Остаток

remainder = pot % winners.size();

4.11.3 Правило

- остаток получает игрок ближе к дилеру;

4.12 Сетевое взаимодействие

4.12.1 Архитектура

- клиент-сервер

4.12.2 Протокол

- TCP

4.12.3 Формат

- JSON

4.13 Синхронизация

- сервер — главный источник данных
- клиент — только отображение

📄 5. Архитектура системы

5.1 Общая архитектура

Программное изделие реализуется по **клиент-серверной архитектуре**, при которой:

- **сервер** отвечает за:
  - игровую логику;
  - хранение состояния игры;
  - обработку действий игроков;
- **клиент** отвечает за:
  - отображение интерфейса;
  - ввод действий пользователя;
  - передачу данных на сервер;

5.1.1 Общая схема взаимодействия

```
Клиент 1 ─┐
Клиент 2 ─┼──▶ Сервер (GameServer)
Клиент 3 ─┤
Клиент 4 ─┘
```

5.1.2 Принцип работы

1. Клиенты подключаются к серверу;
2. Сервер хранит всех игроков;
3. Сервер управляет игрой;
4. Клиенты получают обновления состояния;
5. Все действия проходят через сервер;

👉 Важно:
сервер является единственным источником истины

5.2 Серверная часть

Серверная часть реализуется как многопоточное приложение на Java.

5.2.1 Основные компоненты сервера

---

1. GameServer

Назначение:

- запуск сервера;
- ожидание подключений;
- создание потоков для клиентов;

Основные функции:

- открытие ServerSocket;
- прием новых соединений;
- создание PlayerSession;

---

2. PlayerSession

Назначение:

- обработка одного клиента;

Функции:

- чтение сообщений;
- отправка ответов;
- передача действий в GameEngine;

---

3. GameTable

Назначение:

- хранение игроков;
- управление одним игровым столом;

Содержит:

- список игроков;
- текущий банк;
- текущий этап игры;

---

4. GameEngine

Назначение:

- реализация всей логики игры;

Функции:

- запуск раундов;
- управление ходами;
- обработка ставок;
- переход между этапами;

---

5. Player

Назначение:

- модель игрока;

Содержит:

- имя;
- баланс;
- карты;

- текущую ставку;
- статус (в игре / fold);

---

6. Deck

Назначение:

- работа с колодой;

Функции:

- создание колоды;
- перемешивание;
- раздача карт;

---

7. Card

Назначение:

- представление карты;

Поля:

- масть (Suit);
- значение (Rank);

---

8. HandEvaluator

Назначение:

- определение комбинаций;

Функции:

- анализ 7 карт;
- поиск лучшей комбинации;

- сравнение комбинаций;

5.3 Клиентская часть

Клиент реализуется как приложение с графическим интерфейсом.

5.3.1 Основные компоненты клиента

---

1. UI (User Interface)

Назначение:

- отображение игры;

Функции:

- вывод карт;
- отображение игроков;
- кнопки действий;

---

2. NetworkClient

Назначение:

- связь с сервером;

Функции:

- подключение;
- отправка сообщений;
- получение обновлений;

---

3. GameController

Назначение:

- логика клиента;

Функции:

- обработка данных от сервера;
- обновление интерфейса;
- управление действиями игрока;

5.4 Потоки выполнения

5.4.1 Сервер

- основной поток:
  - принимает подключения;
- отдельный поток на каждого клиента:
  - обрабатывает сообщения;

5.4.2 Клиент

- поток UI;
- поток сети (чтение данных);

5.5 Взаимодействие компонентов

5.5.1 Сервер

PlayerSession → GameEngine → GameTable
↓
обновление состояния
↓
отправка клиентам

5.5.2 Клиент

NetworkClient → GameController → UI

5.6 Хранение данных

5.6.1 Сервер хранит:

- список игроков;
- текущую игру;
- карты;
- ставки;
- банк;

5.6.2 Клиент хранит:

- своё имя;
- текущее состояние (копия с сервера);

5.7 Расширяемость системы

Архитектура предусматривает возможность расширения:

- добавление нескольких столов;
- добавление авторизации;
- добавление ботов;
- добавление базы данных;

📄 6. Пользовательский интерфейс

6.1 Общая характеристика интерфейса

Пользовательский интерфейс реализуется в виде графического приложения на языке Java с использованием библиотек **JavaFX** или **Swing**.

Интерфейс должен обеспечивать:

- удобство взаимодействия пользователя с системой;
- наглядное отображение игрового процесса;
- быстрый доступ к игровым действиям;
- минимальное количество лишних элементов.

Основные требования:

- интуитивная понятность;
- логичное расположение элементов;
- оперативное обновление состояния игры;
- визуальное выделение активного игрока.

6.2 Экран подключения

6.2.1 Назначение

Экран подключения предназначен для:

- ввода имени пользователя;
- подключения к серверу;
- отображения ошибок подключения.

6.2.2 Элементы интерфейса

На экране должны присутствовать:

- поле ввода имени;
- кнопка «Подключиться»;
- текстовое поле для отображения ошибок;
- (опционально) поле ввода адреса сервера и порта;

6.2.3 Поведение элементов

- кнопка «Подключиться» активна только при корректном имени;
- при нажатии:
  - выполняется попытка подключения;
  - при успехе — переход к игровому экрану;
  - при ошибке — отображается сообщение;

6.2.4 Сообщения об ошибках

Возможные сообщения:

- «Некорректное имя пользователя»;
- «Сервер недоступен»;
- «Превышено максимальное количество игроков»;

- «Имя уже занято»;

6.3 Игровой экран

6.3.1 Общая структура

Игровой экран делится на несколько областей:

```
| Игрок (верх) |
| |
| Карты стола (центр) |
| |
| Игрок | Пот | Игрок |
| |
| Карты игрока (низ) |
| Кнопки действий |
```

6.3.2 Отображение игроков

Для каждого игрока отображается:

- имя;
- текущий баланс;
- текущая ставка;
- статус:
  - в игре;
  - сбросил карты (Fold);
  - текущий ход;

6.3.3 Отображение карт

Карты игрока:

- отображаются в нижней части экрана;
- видны только текущему игроку;

Карты стола:

- отображаются в центре;
- открываются поэтапно:
  - 3 карты (флоп);
  - 4 карта (тёрн);
  - 5 карта (ривер);

6.3.4 Отображение банка

- отображается в центре экрана;
- содержит текущее значение pot;
- обновляется после каждого действия;

6.3.5 Отображение этапа игры

Игроку должно быть видно:

- текущий этап (Префлоп, Флоп, Тёрн, Ривер, Шоудаун);

6.4 Элементы управления

6.4.1 Кнопки действий

На экране должны присутствовать следующие кнопки:

- Fold
- Check
- Call
- Raise
- All-in

6.4.2 Поле ввода ставки

Для действия Raise:

- поле ввода суммы;
- или слайдер выбора суммы;

6.4.3 Доступность кнопок

Кнопки:

- активны только в момент хода игрока;
- деактивированы в остальное время;

6.4.4 Логика отображения

- Check отображается только при отсутствии ставки;
- Call отображается при наличии ставки;
- Raise доступен при наличии достаточного баланса;

6.5 Таймер хода

6.5.1 Отображение

- отображается рядом с активным игроком;
- визуально представлен в виде:
  - числового таймера;
  - или прогресс-бара;

6.5.2 Поведение

- запускается при начале хода;
- уменьшается в реальном времени;
- при достижении 0 выполняется автоматическое действие;

6.6 Обновление интерфейса

6.6.1 Источник данных

Все данные поступают от сервера.

6.6.2 Обновление

Интерфейс обновляется:

- после каждого действия;
- при смене этапа;
- при изменении состояния игроков;

6.6.3 Синхронизация

- клиент не изменяет данные самостоятельно;
- все изменения происходят через сервер;

6.7 Отображение результатов

6.7.1 Шоудаун

После окончания игры:

- отображаются карты всех игроков;
- показываются комбинации;
- выделяется победитель;

6.7.2 Победитель

- визуально выделяется (цветом/рамкой);
- отображается выигранная сумма;

6.8 Обработка выхода игрока

6.8.1 Визуализация

Если игрок покинул игру:

- его статус обновляется;
- отображается сообщение;

6.8.2 Поведение

- игрок удаляется из интерфейса;
- игра продолжается при достаточном количестве игроков;

📄 7. Нефункциональные требования

7.1 Общие положения

Нефункциональные требования определяют характеристики качества программного изделия, включая:

- производительность;
- надежность;
- удобство использования;
- масштабируемость;
- безопасность.

Данные требования обязательны для обеспечения корректной и стабильной работы системы.

7.2 Требования к производительности

7.2.1 Время отклика

Система должна обеспечивать:

- время реакции на действие пользователя — не более **200 мс** при нормальной нагрузке;
- обновление состояния игры — не более **500 мс** после действия игрока;

7.2.2 Обработка запросов

Сервер должен:

- корректно обрабатывать одновременные действия до 4 игроков;
- не допускать блокировки основного игрового потока;

7.2.3 Нагрузка

Система рассчитана на:

- максимум 4 одновременных клиента;
- один игровой стол;

7.2.4 Использование ресурсов

Программа должна:

- эффективно использовать оперативную память;
- не создавать утечек памяти;
- не перегружать процессор при нормальной работе;

7.3 Требования к надежности

7.3.1 Устойчивость работы

Система должна:

- корректно функционировать при длительной работе;
- не зависать при обработке действий игроков;

7.3.2 Обработка ошибок

Программа должна:

- обрабатывать некорректные входные данные;
- предотвращать выполнение недопустимых действий;
- защищаться от ошибок пользователя;

7.3.3 Работа при сбоях

При сбоях:

- разрыв соединения не должен приводить к краху сервера;
- отключившийся игрок должен быть корректно удален;

7.3.4 Таймауты

Система должна:

- контролировать время хода игрока;
- выполнять автоматические действия при превышении времени;

7.4 Требования к удобству использования (Usability)

7.4.1 Интерфейс

Интерфейс должен быть:

- понятным;
- простым в использовании;
- логично организованным;

7.4.2 Взаимодействие

Пользователь должен:

- быстро понимать доступные действия;
- легко ориентироваться в игровом процессе;

7.4.3 Обратная связь

Система должна:

- отображать текущие действия;
- уведомлять об ошибках;
- информировать о ходе игры;

7.5 Требования к масштабируемости

7.5.1 Возможность расширения

Архитектура должна позволять:

- добавление новых функций;
- увеличение количества столов (в будущем);
- добавление ботов;

7.5.2 Модульность

Система должна быть:

- разделена на независимые модули;
- легко модифицируемой;

7.6 Требования к безопасности

7.6.1 Защита от некорректных действий

Сервер должен:

- проверять все действия игроков;
- не доверять клиентским данным;

7.6.2 Валидация данных

Все входные данные:

- проверяются на корректность;
- фильтруются перед обработкой;

7.6.3 Сетевое взаимодействие

- используется базовое TCP-соединение;
- защита данных упрощена (в рамках учебного проекта);

7.7 Требования к переносимости

7.7.1 Поддерживаемые платформы

Программа должна работать на:

- Windows;
- Linux;

7.7.2 Зависимости

- использование стандартной Java;
- отсутствие необходимости в сторонних библиотеках;

7.8 Требования к сопровождению

7.8.1 Поддерживаемость кода

Код должен:

- быть структурированным;
- соответствовать принципам ООП;
- быть читаемым;

7.8.2 Документирование

Программа должна сопровождаться:

- комментариями в коде;
- описанием архитектуры;

📄 8. Обработка ошибок и исключений

8.1 Общие положения

Система должна обеспечивать:

- стабильную работу при ошибках пользователя;
- устойчивое функционирование при сбоях сети;
- корректное завершение операций при исключениях;

- информирование пользователя о возникших ошибках.

Обработка ошибок выполняется как на **клиенте**, так и на **сервере**.

8.2 Ошибки ввода пользователя

8.2.1 Некорректное имя

- Имя меньше 3 или больше 16 символов;
- Наличие запрещённых символов;
- Пустое поле;
- Дублирование имени среди текущих игроков.

Реакция системы:

- на клиенте: отображается сообщение «Некорректное имя» или «Имя уже занято»;
- на сервере: отклоняет подключение с соответствующим уведомлением;
- ход не продолжается до корректного ввода.

8.2.2 Некорректная ставка

- Ставка меньше минимальной;
- Ставка больше текущего баланса;
- Некорректный формат данных (не число).

Реакция системы:

- сервер отклоняет действие;
- клиент получает уведомление и должен повторить ход;
- таймер хода продолжает работать.

8.3 Ошибки сетевого соединения

8.3.1 Потеря соединения с сервером

- клиент уведомляется о разрыве соединения;
- игра останавливается для отключившегося игрока;
- при возможности выполняется автоматический Fold.

8.3.2 Недоступность сервера

- клиент не может подключиться;
- отображается сообщение «Сервер недоступен»;
- игра не начинается до восстановления соединения.

8.4 Исключения при работе сервера

- **IOException** при чтении/записи сообщений;
- **NullPointerException** при работе с объектами игроков;
- **ConcurrentModificationException** при изменении списков игроков во время итерации.

Реакция системы:

- исключение логируется в журнал;
- сервер продолжает работу для остальных игроков;
- игрок с ошибкой исключается из текущей раздачи;
- при необходимости уведомляются оставшиеся игроки.

8.5 Логирование ошибок

Система должна вести журнал:

- подключения и отключения игроков;
- ошибки ввода;
- таймауты ходов;

- системные исключения.

Журнал предназначен для анализа и отладки.

8.6 Автоматические действия при ошибках

- Превышение таймера хода → автоматический Check или Fold;
- Некорректная ставка → игрок повторяет действие;
- Потеря соединения → автоматический Fold и удаление игрока.

8.7 Проверка корректности данных

- Сервер проверяет все действия клиента;
- Клиент проверяет ввод пользователя;
- Исключения обрабатываются локально и не приводят к аварийному завершению игры.

📄 9. Тестирование

9.1 Общие положения

Цель тестирования:

- Проверка соответствия системы функциональным и нефункциональным требованиям.
- Обнаружение ошибок в логике игры, взаимодействии клиента и сервера, пользовательском интерфейсе.
- Подтверждение стабильности, надежности и корректности работы всех компонентов.

Тестирование выполняется на **этапе разработки** и перед **финальной сдачей проекта**.

9.2 Виды тестирования

9.2.1 Функциональное тестирование

Проверяется:

- корректность раздачи карт;
- правильность работы всех действий игрока (Fold, Check, Call, Raise, All-in);
- соблюдение правил очередности ходов;
- корректность работы таймера;
- правильное определение комбинаций и победителя;
- корректное распределение банка при делении на нескольких игроков;
- обработка отключений игроков;
- правильность обработки ошибок ввода.

9.2.2 Нефункциональное тестирование

Проверяется:

- производительность и время отклика (не более 500 мс для обновления состояния);
- устойчивость работы при длительном сеансе игры;
- корректность работы при потере сетевого соединения;
- нагрузка: работа при 2–4 игроках одновременно;
- безопасность: проверка валидации всех входных данных.

9.2.3 Юзабилити-тестирование

Проверяется:

- интуитивность интерфейса;

- корректность отображения карт и игроков;
- наглядность таймера и банка;
- корректная визуализация победителя.

9.2.4 Модульное тестирование

Проверяется:

- классы Card и Deck: корректность создания колоды и перемешивания;
- класс HandEvaluator: правильность определения комбинаций;
- GameEngine: логика этапов игры;
- PlayerSession: корректная обработка действий игрока.

9.2.5 Интеграционное тестирование

Проверяется:

- взаимодействие клиента и сервера;
- обновление состояния игры на клиенте после действий игроков;
- корректная синхронизация данных между всеми клиентами.

9.3 Сценарии тестирования

Примеры сценариев:

1. **Подключение игроков**
    - Два игрока подключаются, вводят корректные имена.
    - Проверка корректного отображения на клиенте.
2. **Раздача карт**
    - Каждому игроку раздаются две карты.

- На стол выкладываются карты поэтапно (флоп, тёрн, ривер).

3. **Ход игрока**
   - Игрок делает допустимое действие (Call, Raise, Fold, Check).
   - Проверка обновления банка и очередности ходов.
4. **Таймер хода**
   - Игрок не делает ход 20 секунд.
   - Система выполняет автоматический Check/Fold.
5. **All-in и распределение банка**
   - Игрок ставит весь баланс.
   - Проверка корректного распределения при целочисленном и нецелочисленном делении.
6. **Обработка ошибок**
   - Ввод некорректного имени.
   - Ввод ставки больше баланса.
   - Потеря соединения клиента.
7. **Определение победителя**
   - Проверка комбинаций на корректность.
   - Проверка распределения банка между победителями.

9.4 Инструменты тестирования

- **JUnit** для модульного тестирования Java-классов.
- **Логи сервера** для анализа действий игроков и обработки ошибок.
- **Пошаговое наблюдение на клиенте** для проверки визуальной части.
- **Сетевые тесты** с эмуляцией нескольких клиентов.

9.5 Критерии приемки

Система считается готовой к сдаче, если:

- все функциональные требования выполнены;
- ошибки критического уровня отсутствуют;
- интерфейс корректно отображает все элементы;
- обработка таймеров, ставок и комбинаций работает корректно;
- все тестовые сценарии пройдены успешно.

📄 10. Возможные расширения и дальнейшее развитие проекта

10.1 Добавление нескольких столов

- Реализация поддержки нескольких игровых столов одновременно.
- Возможность создания новых комнат для игры.
- Поддержка 2–4 игроков на каждом столе.
- Сервер управляет отдельными экземплярами `GameTable` для каждого стола.
- Клиент получает список доступных столов и может подключиться к выбранному.

10.2 Добавление авторизации и профилей игроков

- Регистрация пользователей с логином и паролем.
- Поддержка уникальных идентификаторов игроков.
- Хранение статистики игры (победы, проигрыши, баланс).

- Возможность входа с разных устройств, синхронизация состояния.

10.3 Добавление ботов

- Разработка искусственного интеллекта для игры за пустые места.
- Настройка уровня сложности (легкий, средний, сложный).
- Поддержка всех игровых действий (Fold, Check, Call, Raise, All-in).
- Серверная логика определяет ходы бота и синхронизирует с остальными игроками.

10.4 Расширение интерфейса клиента

- Анимации карт и действий.
- Звуковое сопровождение (например, звук раздачи карт, победы, сброса).
- Настройка визуальной темы игры.
- Поддержка сенсорного управления (для мобильной версии).

10.5 Подключение к базе данных

- Хранение статистики игроков и истории игр.
- Возможность повторного анализа игр.
- Поддержка сохранения баланса и прогресса между сессиями.

10.6 Реализация многоплатформенной версии

- Клиент может работать на Windows, Linux и macOS.
- Возможность мобильной версии на Android или iOS.
- Поддержка web-клиента через WebSocket для браузера.

10.7 Развитие игровой логики

- Добавление новых видов покера (например, Омаха, Seven-Card Stud).
- Настройка правил игры (лимит ставок, структура банка).
- Поддержка турниров и лиг с автоматическим подсчетом рейтинга.

10.8 Интеграция с внешними сервисами

- Возможность подключения платежных систем (только для учебного проекта — демонстрация).
- Подключение чат-сервиса для общения игроков.
- Лидеры и статистика в реальном времени.

10.9 Масштабирование сервера

- Возможность балансировки нагрузки при увеличении числа игроков.
- Поддержка горизонтального масштабирования: несколько серверов для одного игрового пространства.
- Использование очередей сообщений для синхронизации действий игроков.

10.10 Возможные дополнительные улучшения

- Темы оформления (светлая/темная).
- Встроенные подсказки для новичков.
- Система достижений и наград.
- Настройка таймера хода по желанию игрока.